

Activity 01

```
-- Create and select the new database
CREATE DATABASE shop_db;
USE shop_db;

-- Create the Customers table
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50),
    city VARCHAR(50)
);

-- Create the Orders table
CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    customer_id INT,
    product_name VARCHAR(50),
    amount DECIMAL(10, 2)
);

-- Insert data into the Customers table
INSERT INTO customers (customer_id, customer_name, city) VALUES
(1, 'Kamal', 'Colombo'),
(2, 'Nimal', 'Kandy'),
(3, 'Sunil', 'Galle'),
(4, 'Piyal', 'Matara');

-- Insert data into the Orders table
INSERT INTO orders (order_id, customer_id, product_name, amount) VALUES
(101, 1, 'Laptop', 150000.00),
(102, 1, 'Mouse', 1500.00),
(103, 2, 'Keyboard', 3500.00),
(104, 5, 'Monitor', 45000.00);
```

Section 1: Practice Questions

Activity 1: Basic INNER JOIN Write an SQL query to display the names of the customers (customer_name) who have placed an order, along with the names of the products (product_name) they purchased.

```
select
    customers.customer_name,
    orders.product_name
from customers
inner join orders on customers.customer_id =
orders.customer_id;
```

Activity 2: Using a LEFT JOIN Write an SQL query to display **all** customers in our shop and the products they bought. If a customer hasn't bought anything, they must still appear in the list.

```
select
    customers.customer_name,
    orders.product_name
from customers
left join orders on customers.customer_id =
orders.customer_id;
```

Activity 3: Using a RIGHT JOIN Write an SQL query to show **every order** placed in the shop and the name of the customer who placed it. Even if an order has a missing or invalid customer record (due to a system error), the order must still be shown in the list.

```
select
    customers.customer_name,
    orders.product_name
from customers
right join orders on customers.customer_id =
orders.customer_id;
```

Activity 4: Logical Thinking (JOIN with a WHERE clause) Write an SQL query to find out which registered customers have **not placed any orders yet**. (Hint: Think about which join to use and how to check for empty values).

```
select
    customers.customer_name
from customers
left join orders on customers.customer_id =
orders.customer_id
where orders.order_id is null;
```

Activity 5: Adding Conditions Write an SQL query to find the names, cities, and purchased products of customers who placed an order with an amount **greater than 10,000**.

Activity 02

Database Concept: Company Database

Table 1: employees

emp_id	emp_name	dept_id
1	Alice	101 ▾
2	Bob	102 ▾
3	Charlie	102 ▾
4	David	103 ▾
5	Eve	NULL ▾

Table 2: departments

dept_id	dept_name
101	HR
102	IT
103	Finance
104	Marketing

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT  
);
```

```
CREATE DATABASE company_db;  
USE company_db;
```

```
CREATE TABLE departments (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50)  
);
```

```
CREATE TABLE employees (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    dept_id INT  
);
```

```
INSERT INTO departments (dept_id, dept_name) VALUES
(101, 'HR'), (102, 'IT'), (103, 'Finance'), (104, 'Marketing');

INSERT INTO employees (emp_id, emp_name, dept_id) VALUES
(1, 'Alice', 101), (2, 'Bob', 102), (3, 'Charlie', 102), (4, 'David', 103), (5,
'Eve', NULL);
```

Activity Questions

Please write the SQL queries for the following questions based on the tables above.

Question 1: Write an SQL query to display the names of all employees who are assigned to a department, along with the names of their departments.

```
select
    employees.emp_name,
    departments.dept_name
from employees
join departments on employees.dept_id =
departments.dept_id;
```

Question 2: Write an SQL query to list **ALL** employees and their department names. If an employee does not have a department yet, their name should still appear on the list.

```
select
    employees.emp_name,
    departments.dept_name
from employees
left join departments on employees.dept_id =
departments.dept_id;
```

Question 3: Write an SQL query to display **ALL** departments and the names of the employees working in them. Even if a department has no employees, it must still be shown in the results.

```
select
    employees.emp_name,
    departments.dept_name
```

```
from employees
right join departments on employees.dept_id =
departments.dept_id;
```

Question 4: (Logical Filter) Write an SQL query to find the names of the employees who have **NOT** been assigned to any department yet.

```
select
    employees.emp_name
from employees
left join departments on employees.dept_id =
departments.dept_id
where employees.dept_id is null;
```

Question 5: (Logical Filter) Write an SQL query to find the names of the departments that currently have **NO** employees assigned to them.

```
select
    departments.dept_name
from employees
right join departments on employees.dept_id =
departments.dept_id
where employees.dept_id is null;
```

Activity 03

Database Concept: Library System

```
-- Create and select the database
CREATE DATABASE library_db;
USE library_db;

-- Create the Members table
CREATE TABLE members (
    member_id INT PRIMARY KEY,
    member_name VARCHAR(50)
```

```

);

-- Create the Books table
-- (member_id indicates who has currently borrowed the book)
CREATE TABLE books (
    book_id INT PRIMARY KEY,
    book_title VARCHAR(100),
    member_id INT
);

-- Insert data into the Members table
INSERT INTO members (member_id, member_name) VALUES
(1, 'Saman'),
(2, 'Ruwan'),
(3, 'Nayana'),
(4, 'Kasun');
-- Note: Kasun has not borrowed any books.

-- Insert data into the Books table
INSERT INTO books (book_id, book_title, member_id) VALUES
(101, 'Madol Doova', 1),
(102, 'Harry Potter', 2),
(103, 'Sherlock Holmes', 1),
(104, 'The Hobbit', NULL),
-- Note: The Hobbit is available in the library (not borrowed).
(105, 'Advanced SQL', 99);
-- Note: Member 99 does not exist in the members table (System Error/Lost
record).

```

Section 1: Practice Questions

Please provide these questions to your students to practice.

Question 1: Write an SQL query to list the names of the members and the titles of the books they have currently borrowed.

Question 2: Write an SQL query to show **ALL** registered members and the books they have borrowed. If a member hasn't borrowed any books (like Kasun), their name should still appear on the list.

Question 3: Write an SQL query to display **ALL** the books in the library system, along with the names of the members who borrowed them. If a book is currently available and not borrowed by anyone, it must still be shown in the results.

Question 4: (Logical Filter - Finding Available Items) Write an SQL query to find the titles of the books that are currently **available** in the library (i.e., books that have not been borrowed by anyone).

Question 5: (Logical Filter - Finding Inactive Users) Write an SQL query to find the names of the members who have **NOT** borrowed any books from the library.

Activity 03

Database Concept: School Clubs System

```
CREATE DATABASE school_clubs_db;
USE school_clubs_db;

CREATE TABLE clubs (
    club_id INT PRIMARY KEY,
    club_name VARCHAR(50)
);

CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(50),
    club_id INT
);

INSERT INTO clubs (club_id, club_name) VALUES
(101, 'Science Club'),
(102, 'Drama Club'),
(103, 'Sports Club'),
(104, 'Art Club');

INSERT INTO students (student_id, student_name, club_id) VALUES
(1, 'Nimal', 101),
(2, 'Kamal', 102),
(3, 'Amal', 103),
(4, 'Sunimal', 101),
(5, 'Ruwani', NULL);
```

Section 1: Practice Questions

Please write the SQL queries for the following 10 questions using *INNER*, *LEFT*, *RIGHT*, or *FULL* joins.

Basic Joins

1. Write an SQL query using an **INNER JOIN** to display the names of the students and the names of the clubs they have joined.

```
select
    students.student_name,
    clubs.club_name
```

```
from students
join clubs on students.club_id = clubs.club_id;
```

2. Write an SQL query using a **LEFT JOIN** to list **ALL** students and their club names. If a student has not joined a club, they must still be on the list.

```
select
    students.student_name,
    clubs.club_name
from students
left join clubs on students.club_id =
clubs.club_id;
```

3. Write an SQL query using a **RIGHT JOIN** to display **ALL** clubs and the students in them. Even if a club has no students, it must appear on the list.

```
select
    students.student_name,
    clubs.club_name
from students
right join clubs on students.club_id =
clubs.club_id;
```

4. Write an SQL query to perform a **FULL OUTER JOIN** (Use UNION of LEFT and RIGHT joins for MySQL) to show all students and all clubs, including unmatched students and unmatched clubs.

```
select
    students.student_name,
    clubs.club_name
from students
left join clubs on students.club_id = clubs.club_id
union
select
```

```
students.student_name,  
clubs.club_name  
from students  
right join clubs on students.club_id =  
clubs.club_id;
```

Adding Conditions

5. Write an SQL query using an **INNER JOIN** to find only the students who have joined the 'Science Club'.

```
select  
students.student_name,  
clubs.club_name  
from students  
join clubs on students.club_id = clubs.club_id  
where clubs.club_name = 'Science Club';
```

6. Write an SQL query using a **LEFT JOIN** to find the names of the students who have **NOT** joined any club yet. (Hint: Use WHERE IS NULL).

7. Write an SQL query using a **RIGHT JOIN** to find the names of the clubs that currently have **NO** students enrolled in them.

8. Write an SQL query using an **INNER JOIN** to list the students and clubs, but only for the 'Drama Club' and 'Sports Club'.

```
select  
students.student_name,  
clubs.club_name  
from students  
join clubs on students.club_id = clubs.club_id  
where clubs.club_name = 'Drama club' or  
clubs.club_name = 'Sports Club';
```

Sorting and Advanced Logic

9. Write an SQL query using a **LEFT JOIN** to show all students and their clubs, but sort the final result alphabetically by the student's name (ORDER BY).

```
select
    students.student_name,
    clubs.club_name
from students
left join clubs on students.club_id = clubs.club_id
order by students.student_name asc;
```

10. Write an SQL query using a **FULL OUTER JOIN** (with UNION) to find **ONLY** the unmatched records: students who have no club **AND** clubs that have no students. (Hint: Add WHERE IS NULL conditions to both halves of the UNION).

```
////////////////////////////////////////////////////
```

Question 1: The Ride-Sharing App (Uber/PickMe Scenario)

Context: You are managing the database for a new ride-sharing application. You have a table for registered drivers and a table for rides completed today.

Database Tables:

- **drivers table:** driver_id, driver_name (Data: 1-Kamal, 2-Nimal, 3-Sunil)
- **rides table:** ride_id, driver_id, destination, fare (Data: Kamal and Nimal have completed rides. Sunil has not driven today. There is one ride record with a missing driver id due to a system glitch).

```
-- Create database for Ride-Sharing App
CREATE DATABASE rideshare_db;
USE rideshare_db;

-- Create drivers table
CREATE TABLE drivers (
    driver_id INT PRIMARY KEY,
    driver_name VARCHAR(50)
);

-- Create rides table
```

```

CREATE TABLE rides (
    ride_id INT PRIMARY KEY,
    driver_id INT,
    destination VARCHAR(100),
    fare DECIMAL(10, 2)
);

-- Insert data into drivers
INSERT INTO drivers (driver_id, driver_name) VALUES
(1, 'Kamal'),
(2, 'Nimal'),
(3, 'Sunil');
-- Note: Sunil will have 0 rides.

-- Insert data into rides
INSERT INTO rides (ride_id, driver_id, destination, fare) VALUES
(101, 1, 'Colombo', 1500.00),
(102, 1, 'Gampaha', 2000.00),
(103, 2, 'Kandy', 3500.00),
(104, NULL, 'Galle', 4500.00);
-- Note: Ride 104 has a missing driver_id (System Glitch).

```

The Task / Question: The company wants to give a warning to drivers who registered but **did not complete any rides today**. Write an SQL query to find the names of the drivers who have exactly **ZERO** rides recorded for today.

```

select
    drivers.driver_name
from drivers
left join rides on drivers.driver_id =
rides.driver_id
where rides.ride_id is null;

```

Question 2: The Restaurant Menu (Food Delivery App)

Context: You are building the database for a restaurant menu. The menu is divided into categories (like Burgers, Drinks, Desserts), and food items are linked to these categories.

```

-- Create database for Restaurant Menu

```

```

CREATE DATABASE restaurant_db;
USE restaurant_db;

-- Create categories table
CREATE TABLE categories (
    category_id INT PRIMARY KEY,
    category_name VARCHAR(50)
);

-- Create food_items table
CREATE TABLE food_items (
    item_id INT PRIMARY KEY,
    item_name VARCHAR(100),
    price DECIMAL(10, 2),
    category_id INT
);

-- Insert data into categories
INSERT INTO categories (category_id, category_name) VALUES
(10, 'Burgers'),
(20, 'Drinks'),
(30, 'Desserts');

-- Note: 'Desserts' category will remain empty.

-- Insert data into food_items
INSERT INTO food_items (item_id, item_name, price, category_id) VALUES
(1, 'Chicken Burger', 800.00, 10),
(2, 'Beef Burger', 1000.00, 10),
(3, 'Cola', 200.00, 20),
(4, 'Orange Juice', 300.00, 20);

```

Database Tables:

- **categories table:** category_id, category_name (Data: 10-Burgers, 20-Drinks, 30-Desserts)
- **food_items table:** item_id, item_name, price, category_id (Data: 'Chicken Burger' is linked to 10. 'Cola' is linked to 20. The 'Desserts' category currently has no items inside it).

The Task / Question: The restaurant owner wants to review the entire menu. Write an SQL query to display **every category name** alongside the **names of the food items** in them. Even if a category (like Desserts) is completely empty and has no food items, the category name must still appear on the final report.

```

select
    categories.category_name,
    food_items.item_name

```

```

from categories
left join food_items on categories.category_id =
food_items.category_id;
-- or
select
    categories.category_name,
    food_items.item_name
from food_items
right join categories on food_items.category_id =
categories.category_id;

```

Question 3: The Hospital System (Finding Unassigned Data)

Context: A hospital manages its operations using a database. There is a table for doctors and a table for admitted patients. Every patient should ideally be assigned to a doctor.

Database Tables:

- **doctors table:** doc_id, doc_name, ward
- **patients table:** patient_id, patient_name, disease, doc_id

The Task / Question: The hospital administrator suspects some data entry errors. They want a report showing two critical things at the same time:

1. All patients, including those who haven't been assigned a doctor yet.
2. All doctors, including those who currently have zero patients assigned to them. Write an SQL query to generate this complete combined list.

```

SELECT patients.patient_name, doctors.doc_name
FROM patients
LEFT JOIN doctors ON patients.doc_id = doctors.doc_id
UNION
SELECT patients.patient_name, doctors.doc_name
FROM patients
RIGHT JOIN doctors ON patients.doc_id = doctors.doc_id;

```